In [16]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


def train_test_split_np(X, y, test_size=0.2, random_state=42):

    rng = np.random.default_rng(random_state)
    idx = np.arange(X.shape[0])
    rng.shuffle(idx)

    test_count = int(X.shape[0] * test_size)
    test_idx = idx[:test_count]
    train_idx = idx[test_count:]

    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]


def sigmoid(z):
    z = np.clip(z, -50, 50)
    return 1.0 / (1.0 + np.exp(-z))


def sigmoid_derivative(a):
    return a * (1.0 - a)
```

In [17]:
```
class MLPBinary:

    def __init__(self, input_size, hidden1=32, hidden2=16, lr=0.1, random_stat

        rng = np.random.default_rng(random_state)
        self.lr = lr

        self.W1 = rng.normal(0, 0.1, size=(input_size, hidden1))
        self.b1 = np.zeros((1, hidden1))

        self.W2 = rng.normal(0, 0.1, size=(hidden1, hidden2))
        self.b2 = np.zeros((1, hidden2))

        self.W3 = rng.normal(0, 0.1, size=(hidden2, 1))
        self.b3 = np.zeros((1, 1))

    def forward(self, X):

        self.z1 = X @ self.W1 + self.b1
        self.a1 = sigmoid(self.z1)


        self.z2 = self.a1 @ self.W2 + self.b2
        self.a2 = sigmoid(self.z2)


        self.z3 = self.a2 @ self.W3 + self.b3
```

```
        self.y_hat = sigmoid(self.z3)
        return self.y_hat

    def backward(self, X, y):

        m = X.shape[0]
        y = y.reshape(-1, 1)

        dZ3 = self.y_hat - y
        dW3 = (self.a2.T @ dZ3) / m
        db3 = np.sum(dZ3, axis=0, keepdims=True) / m

        dA2 = dZ3 @ self.W3.T
        dZ2 = dA2 * sigmoid_derivative(self.a2)
        dW2 = (self.a1.T @ dZ2) / m
        db2 = np.sum(dZ2, axis=0, keepdims=True) / m

        dA1 = dZ2 @ self.W2.T
        dZ1 = dA1 * sigmoid_derivative(self.a1)
        dW1 = (X.T @ dZ1) / m
        db1 = np.sum(dZ1, axis=0, keepdims=True) / m

        self.W3 -= self.lr * dW3
        self.b3 -= self.lr * db3
        self.W2 -= self.lr * dW2
        self.b2 -= self.lr * db2
        self.W1 -= self.lr * dW1
        self.b1 -= self.lr * db1

    def fit(self, X, y, epochs=2000, early_stopping_patience=100):

        y = y.reshape(-1, 1)
        eps = 1e-12
        history = []

        best_loss = np.inf
        patience_counter = 0
        best_weights = None

        for epoch in range(1, epochs + 1):

            y_hat = self.forward(X)
            y_hat = np.clip(y_hat, eps, 1 - eps)

            loss = -np.mean(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))
            history.append(loss)

            self.backward(X, y)

            if loss < best_loss:
                best_loss = loss
                patience_counter = 0
                best_weights = (self.W1.copy(), self.b1.copy(),
```

```
                                self.W2.copy(), self.b2.copy(),
                                self.W3.copy(), self.b3.copy())
            else:
                patience_counter += 1
                if patience_counter >= early_stopping_patience:
                    print(f"\nРанняя остановка на эпохе {epoch} ")
                    self.W1, self.b1, self.W2, self.b2, self.W3, self.b3 = bes
                    break



        return history

    def predict_proba(self, X):
        return self.forward(X).ravel()

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)
```

In [18]:

```
def confusion_elements(y_true, y_pred):

    y_true = np.asarray(y_true).astype(int)
    y_pred = np.asarray(y_pred).astype(int)

    tp = np.sum((y_true == 1) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))

    return tp, tn, fp, fn


def classification_metrics(y_true, y_pred):

    tp, tn, fp, fn = confusion_elements(y_true, y_pred)
    eps = 1e-12

    accuracy = (tp + tn) / (tp + tn + fp + fn + eps)
    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)
    f1 = 2 * precision * recall / (precision + recall + eps)

    return accuracy, precision, recall, f1


def roc_curve_manual(y_true, y_score, points=300):

    thresholds = np.linspace(0, 1, points)
    tpr_list = []
    fpr_list = []

    for t in thresholds:
        y_pred = (y_score >= t).astype(int)
```

```
        tp, tn, fp, fn = confusion_elements(y_true, y_pred)

        tpr = tp / (tp + fn + 1e-12)
        fpr = fp / (fp + tn + 1e-12)
        tpr_list.append(tpr)
        fpr_list.append(fpr)

    fpr = np.array(fpr_list)
    tpr = np.array(tpr_list)
    order = np.argsort(fpr)

    return fpr[order], tpr[order]


def auc_score(fpr, tpr):

    return float(np.sum((np.diff(fpr) * (tpr[1:] + tpr[:-1]) / 2)))


def normalize(X):

    return (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-12)
```

In [ ]:

```
def main():
    url = "https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/
    columns = [
        "class", "cap-shape", "cap-surface", "cap-color", "bruises", "odor",
        "gill-attachment", "gill-spacing", "gill-size", "gill-color",
        "stalk-shape", "stalk-root", "stalk-surface-above-ring",
        "stalk-surface-below-ring", "stalk-color-above-ring",
        "stalk-color-below-ring", "veil-type", "veil-color", "ring-number",
        "ring-type", "spore-print-color", "population", "habitat"
    ]
    df = pd.read_csv(url, header=None, names=columns)
    print(f"Датасет загружен: {df.shape[0]} образцов, {df.shape[1]} признаков"

    y = (df["class"] == "p").astype(int).values
    print(f"Распределение классов: съедобные = {np.sum(y == 0)}, ядовитые = {n

    X_df = df.drop(columns=["class"])
    X = pd.get_dummies(X_df, dtype=float).values
    print(f"После one-hot: {X.shape[1]} признаков")

    X = normalize(X)
    X_train, X_test, y_train, y_test = train_test_split_np(X, y, test_size=0.2
    print(f"Обучающая выборка: {X_train.shape[0]} образцов")
    print(f"Тестовая выборка: {X_test.shape[0]} образцов")

    model = MLPBinary(
        input_size=X_train.shape[1],
        hidden1=32,
        hidden2=16,
        lr=0.1,
```

```
        random_state=42,
    )

    history = model.fit(X_train, y_train, epochs=2000, early_stopping_patience

    y_proba = model.predict_proba(X_test)
    y_pred = model.predict(X_test, threshold=0.5)

    accuracy, precision, recall, f1 = classification_metrics(y_test, y_pred)

    print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1-score: {f1:.4f}")

    fpr, tpr = roc_curve_manual(y_test, y_proba, points=300)
    auc = auc_score(fpr, tpr)

    tp, tn, fp, fn = confusion_elements(y_test, y_pred)

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    axes[0, 0].plot(history, linewidth=1, alpha=0.7, color='blue')
    window = 20
    if len(history) > window:
        smoothed = np.convolve(history, np.ones(window)/window, mode='valid')
        axes[0, 0].plot(range(window-1, len(history)), smoothed,
                        linewidth=2, color='red', label=f'Сглаженная (окно={wir
    axes[0, 0].set_xlabel('Эпоха')
    axes[0, 0].set_ylabel('Loss (Бинарная кросс-энтропия)')
    axes[0, 0].set_title('Функция потерь в процессе обучения')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)

    axes[0, 1].plot(fpr, tpr, linewidth=2, label=f'MLP (AUC = {auc:.4f})', col
    axes[0, 1].plot([0, 1], [0, 1], 'k--', label='Случайный классификатор', li
    axes[0, 1].fill_between(fpr, tpr, alpha=0.2, color='blue')
    axes[0, 1].set_xlabel('False Positive Rate (FPR)')
    axes[0, 1].set_ylabel('True Positive Rate (TPR)')
    axes[0, 1].set_title('ROC-кривая')
    axes[0, 1].legend(loc='lower right')
    axes[0, 1].grid(True, alpha=0.3)

    conf_matrix = np.array([[tn, fp], [fn, tp]])
    im = axes[1, 0].imshow(conf_matrix, cmap='Blues', interpolation='nearest')
    axes[1, 0].set_xticks([0, 1])
    axes[1, 0].set_yticks([0, 1])
    axes[1, 0].set_xticklabels(['Съедобный', 'Ядовитый'])
    axes[1, 0].set_yticklabels(['Съедобный', 'Ядовитый'])
    axes[1, 0].set_xlabel('Предсказано')
    axes[1, 0].set_ylabel('Факт')
    axes[1, 0].set_title('Матрица ошибок')
    for i in range(2):
```

```
        for j in range(2):
            axes[1, 0].text(j, i, str(conf_matrix[i, j]), ha='center', va='cer
    plt.colorbar(im, ax=axes[1, 0])


    fig.delaxes(axes[1, 1])

    plt.suptitle('Анализ классификации грибов (MLP с двумя скрытыми слоями)',
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    main()
```

```
Датасет загружен: 8124 образцов, 23 признаков
Распределение классов: съедобные = 4208, ядовитые = 3916
После one-hot: 117 признаков
Обучающая выборка: 6500 образцов
Тестовая выборка: 1624 образцов
Accuracy: 1.0000 (100.00%)
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000
```